

Backend Engineering Take-Home Assignment

Duration: 48 hours

At the end of your time, **promptly send** all of your code to [<rockstars@portal26.ai>](mailto:rockstars@portal26.ai) or create a **private github** repository and give access to `makarthikeyan1`, `jitendra-titaniam`, `sunil-titaniam`

Project Overview: High-Throughput Event Receiver Service

As part of our backend infrastructure, we operate a high-volume data ingestion pipeline that serves as the entry point for all customer-generated events. In this exercise, your goal is to build a service that can receive, filter, batch, and store these events in near real-time.

This exercise is an open book. You are free and encouraged to use your favorite IDE, tools, libraries, frameworks, open-source code, and AI coding assistants. We want to evaluate your **ability to solve real-world engineering problems in a realistic setting**, not how well you memorize syntax.

Objectives

You are to implement and deploy a backend service that:

1. Exposes a **public HTTP endpoint** to receive **POST** requests with a body size between **1KB and 10MB**.
 2. Reads a **custom HTTP header** (e.g., **X-Customer-Tier**) from each request.
 3. **Filters** incoming requests by allowing only a predefined set of header values for above header, which are loaded at **service startup** via configuration.
 4. Maintains a **low-latency pipeline** so data reaches S3 **near real time** (within a few seconds ideally).
 5. As a counter-weight to the previous requirement, **reduce the number of S3 writes** as much as possible. This is only to reduce the number of writes, not the size of each write.
 6. Implements **scalability considerations** to handle high throughput (e.g., >100 requests/sec).
 7. Includes **basic observability**: metrics (req/sec, S3 writes/sec, filtered reqs), and logging for success/failure cases.
 8. Can be deployed and tested using **AWS infrastructure** (credentials will be provided).
-

Functional Requirements

- POST endpoint: `/ingest`

Example:

Request

```
POST http://<hostname>/ingest
{
  "event_timestamp": "2024-01-11T01:42:50.234200+00:00",
  "body": "what is the capital of India?"
}
```

Response

```
{
  "status": "success"
}
```

- HTTP header: `X-Customer-Tier`
 - Valid header values: `["free", "pro", "enterprise"]` (loaded at startup)
 - Batch of data to be stored in S3
 - Each batch can contain **multiple events** but should not exceed **5MB per file**
 - Batches should be **flushed either on size threshold or after a max delay of 5 seconds**
-

Infrastructure Expectations

- Use S3 bucket: `<your-name>_portal26_events`
 - Your code must include **README** with:
 - Build and deployment instructions
 - Sample configuration file
 - How to run locally and how to deploy on AWS
 - Dockerfile and Terraform is optional but a plus.
-

Success Criteria

To be considered a *complete submission*, your solution must:

- Is deployed on AWS in the account that is shared.

- Process and store events correctly in S3 as described.
 - Include tests covering core logic.
 - Handle invalid headers gracefully.
 - Clearly log and surface metrics.
 - Demonstrate basic batching and throughput optimization.
-

What to Submit

1. A Private GitHub (or zip) repo with:
 - Source code
 - Dockerfile
 - README with any instructions as required.
 - Give access to private github repository to `makarthikeyan1`, **jitendra-titaniam**, **sunil-titaniam**
 2. Sample output: Sample data stored in S3, logs and metrics
-

Final Notes

- We understand this is a time-boxed exercise. It's okay if you don't build everything perfectly. Focus on designing something that works and demonstrates solid engineering instincts.
 - Make reasonable trade-offs and document them.
 - We value creativity, clarity, and pragmatism over completeness.
 - If you are stuck, make reasonable assumptions and include those in your README.
-

Good luck! We're excited to see how you approach this.
